Eventos React

Programação Web 2

Objetivo de Aprendizagem

- Compreender o uso de eventos na perspectiva do React
- Utilizar eventos em interfaces web

Agenda

- Eventos
- Renderização Condicional
- Listas
- Formulários
- *Hooks*

Eventos React

Eventos React

> Assim como os eventos DOM do HTML, o React pode executar ações com base em eventos do usuário, tais como cliques, alteração de campos de formulário, movimento do mouse, etc.

Adicionando Eventos

- Usar sintaxe `camelCase`
  - `onClisk` ao invés de ~~`onclick`~~
- *Event handlers* devem estar entre `{ }`
  - `onClick={shoot}` ao invés de ~~`onclick="shoot()"`~~

```
1  <button onClick={shoot}>Take the Shot!</button>
```

```
1  function Football() {
2    const shoot = () => {
3      alert("Great Shot!");
4    }
5
6    return (
7      <button onClick={shoot}>Take the shot!</button>
8    );
9  }
10
11 createRoot(document.getElementById('root')).render(
12   <Football />
13 );
```

Passando Argumentos

Argumentos são passados utilizando *arrow functions*

```
1  function Football() {
2    const shoot = (a) => {
3      alert(a);
4    }
5
6    return (
7      <button onClick={() => shoot("Goal!")}>Take the shot!</button>
8    );
9  }
10
11 createRoot(document.getElementById('root')).render(
12   <Football />
13 );
```

React Event Object

> Os manipuladores de eventos têm acesso ao evento React que disparou a função. Em nosso exemplo, o evento é o "click".

```
function Football() {
  const shoot = (a, b) => {
    alert(b.type);
    /*
    'b' represents the React event that triggered the function,
    in this case the 'click' event
    */
  }

  return (
    <button onClick={(event) => shoot("Goal!", event)}>Take the sho
  );
}

createRoot(document.getElementById('root')).render(
```

Renderização Condicional

`if`

Exemplo 1

```
function MissedGoal() {
  return <h1>MISSED!</h1>;
}

function MadeGoal() {
  return <h1>Goal!</h1>;
}
```

Exemplo 1

> O terceiro componente decide qual dos componentes anteriores utilizar

```
1  function Goal(props) {
2    const isGoal = props.isGoal;
3    if (isGoal) {
4      return <MadeGoal />;
5    }
6    return <MissedGoal />;
7  }
8
9  createRoot(document.getElementById('root')).render(
10   <Goal isGoal={false} />
11 );
```

Operador `&&`

> Outra forma de implementar renderização lógica é usando o operador `&&`

Operador `&&`

Exemplo 2

```
1  function Car(props) {
2    return (
3      <>
4        {props.brand && <h1>My car is a {props.brand}!</h1>}
5      </>
6    );
7  }
8
9  createRoot(document.getElementById('root')).render(
10   <Car brand="Ford" />
11 );
```

Veja o exemplo!

Operador Ternário

Exemplo 3

```
1  function Goal(props) {
2    const isGoal = props.isGoal;
3    return (
4      <>
5        { isGoal ? <MadeGoal/> : <MissedGoal/> }
6      </>
7    );
8  }
9
10 createRoot(document.getElementById('root')).render(
11   <Goal isGoal={false} />
12 );
```

Listas

Listas

> Em React as listas são geradas através de laços e `map()`

```
function MyCars() {
  const cars = ['Ford', 'BMW', 'Audi'];
  return (
    <>
      <h1>My Cars:</h1>
      <ul>
        {cars.map((car) => <li>I am a { car }</li>)}
      </ul>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <MyCars />
);
```

Listas React com Chaves

- As chaves permitem que o React rastreie os elementos da lista
- Se um item for atualizado ou removido, apenas esse item será renderizado novamente, em vez da lista inteira
- Chaves devem ser únicas entre os elementos, mas não precisam ser únicas em toda a aplicação

Listas

Exemplo 4

```
1  function MyCars() {
2    const cars = [
3      {id: 1001, brand: 'Ford'},
4      {id: 1002, brand: 'BMW'},
5      {id: 1003, brand: 'Audi'}
6    ];
7    return (
8      <>
9        <h1>My Cars:</h1>
10       <ul>
11         {cars.map((car) => <li key={car.id}>I am a { car.brand }</l
12       </ul>
13     </>
```

Listas

Exemplo 4 modificado

```
1  function MyCars() {
2    const cars = ['Ford', 'BMW', 'Audi'];
3    return (
4      <>
5        <h1>My Cars:</h1>
6        <ul>
7          {cars.map((car, index) => <li key={index}>I am a { car }</l
8        </ul>
9      </>
10   );
11 }
12
13 createRoot(document.getElementById('root')).render(
```

Formulários

HTML versus React

- Em React, elementos de formulário como `<input>`, `<textarea>` e `<select>` funcionam diferentemente do HTML tradicional
  - **HTML**, os elementos de formulário mantêm seu próprio valor com base na entrada do usuário
  - Por exemplo, um campo `<input type="text">` mantém o controle do seu próprio valor no DOM do HTML.
- **React**, o valor do elemento de formulário é mantido na propriedade `state` do componente e atualizado apenas com a função `setState()`

Componentes Controlados

Controlled components

- Dados de formulários são gerenciados pelo componente React
- O valor de `<input>`, por exemplo faz parte do *state*
- Dados gerenciados pelo componente são armazenados no próprio componente
- `useState` deve ser utilizado para cada entrada

Exemplo 5

```
1   import { useState } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function MyForm() {
5     const [name, setName] = useState("");
6
7     function handleChange(e) {
8       setName(e.target.value);
9     }
10
11    return (
12      <form>
13        <label>Enter your name:
14          <input type="text" value={name} onChange={handleChange} />
15        </label>
16        <p>Current value: {name}</p>
17      </form>
18    )
19  }
20
21  createRoot(document.getElementById('root')).render(
```

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function MyForm() {
  const [name, setName] = useState("");

  function handleChange(e) {
    setName(e.target.value);
  }

  function handleSubmit(e) {
    e.preventDefault();
    alert(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Enter your name:
        <input
          type="text"
          value={name}
          onChange={handleChange}
```

Hooks

Hooks

> Hooks permitem que as funções acessem o estado e outros recursos do React sem usar classes

Hooks

- Precisam ser importadas
  - `import { useState } from 'react';`
- `useState` é um *hook* que monitora o estado da aplicação (dados ou propriedades)

Hooks

Regras de uso

- Apenas podem ser chamadas dentro de um componente
- Não podem ser condicionais
- Apenas podem ser chamadas no primeiro nível de um componente
- Não funcionam em componentes de classe

useState

Exemplo

```
1  import { useState } from "react";
2
3  function FavoriteColor() {
4    const [color, setColor] = useState("red");
5  }
```

useState

Lendo o estado

```
1  import { useState } from 'react';
2  import { createRoot } from 'react-dom/client';
3
4  function FavoriteColor() {
5    const [color, setColor] = useState("red");
6
7    return <h1>My favorite color is {color}!</h1>
8  }
9
10 createRoot(document.getElementById('root')).render(
11   <FavoriteColor />
12 );
```

useState

Atualizando o estado

```
1  <button
2    type="button"
3    onClick={() => setColor("blue")}
4  >Blue</button>
```

useState

Objeto *state*

```
1  import { useState } from 'react';
2  import { createRoot } from 'react-dom/client';
3
4  function MyCar() {
5    const [car, setCar] = useState({
6      brand: "Ford",
7      model: "Mustang",
8      year: "1964",
9      color: "red"
10   });
11
12   return (
13     <>
```

`useState`

Atualizando objetos *state*

```
1  const updateColor = () => {
2    setCar(previousState => {
3      return { ...previousState, color: "blue" }
4    });
5  }
```

Outros *Hooks*

- `setEffect`
- `useContext`
- `useRef`
- `useReducer`
- `useCallback`
- `useMemo`
- *Hooks* Personalisadas

> Próxima Aula

Perguntas?

Exercício

1

Crie um componente React que permita ao usuário:

- Adicionar novas tarefas via um formulário com um campo de input e botão de *submit*
- Exibir a lista de tarefas usando `map()` para renderizar cada item
- Marcar tarefas como concluídas ou não concluídas com um evento de clique (ex.: *checkbox* ou botão)
- Remover tarefas com um botão de deletar
- Usar renderização condicional para mostrar apenas tarefas pendentes ou todas, com base em um filtro (ex.: botões "Todas", "Pendentes", "Concluídas").

1

Estrutura básica

```
1   import React, { useState } from 'react';
2
3   function TodoList() {
4     const [tasks, setTasks] = useState([]);
5     const [input, setInput] = useState('');
6     const [filter, setFilter] = useState('all');
7
8     // Funções para adicionar, toggle e remover tarefas
9     // Renderização condicional para o filtro
10    // Lista mapeada com eventos
11
12    return (
13      <div>
14        <form onSubmit={handleAddTask}>
15          <input value={input} onChange={(e) => setInput(e.target.value)} />
16          <button type="submit">Adicionar</button>
17        </form>
18        {/* Filtros e lista */}
19      </div>
20    );
21  }
```

1

Dicas

- Aplique `useState` para gerenciar o estado da lista de tarefas, do `input` do formulário e do filtro
- Implemente eventos como `onSubmit` no formulário, `onClick` nos botões e `onChange` no input.
- Aplique renderização condicional com operador ternário ou `&&` para o filtro e para estilizar tarefas concluídas (ex.: texto riscado).
- Estruture o componente em JSX, com listas renderizadas dinamicamente

1

> Postar os resultados na Atividade ToDo List do Google Classroom

Referências

- React Documentation
- W3Schools React Components
- React DevTools

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

pw2-repo

Powered by Slidev

Cover image by haikei